

Expert-in-the-Loop LLM Tutoring: Designing and Evaluating Instructor Oversight in a Self-Regulated Learning System

Independent Study Progress Report – Spring 2026

MD JONAYED HOSSAIN CHOWDHURY, Kean University, USA

Conversational AI has become a default study companion for many undergraduates, yet the systems they reach for were not built around how learning actually happens. In earlier work we introduced *Study Assist*, an LLM-powered tutor organized around the three phases of self-regulated learning, and showed in a 55-participant evaluation that learners can complete cognitively demanding milestones without losing usability. That study also exposed a structural gap: *Study Assist* had no mechanism for the person who is normally accountable for what a student learns, the instructor, to shape, scope, or oversee what the AI was teaching. This report describes the second phase of the project, in which we extend *Study Assist* into an *Expert-in-the-Loop* (EitL) tutoring platform and lay out the protocol for evaluating it with computer-science faculty. We motivate EitL as a sociotechnical framing in which a domain expert constrains AI behavior on behalf of layman users, summarize the instructor survey and literature review that drove the redesign, describe the new course/topic/module hierarchy and instructor-facing agents we built, and outline a planned think-aloud study with five to ten faculty members in which participants interact with simulated student cohorts inside a mid-semester and a full-semester Java course. The contribution of this report is the integrated design and study plan, which together set up the empirical evaluation that the next phase of the work will execute.

Additional Key Words and Phrases: Large Language Models, Self-Regulated Learning, Expert-in-the-Loop, Human-AI Collaboration, Educational Technology, Instructor Dashboards, Think-Aloud Evaluation

1 Introduction

Most undergraduates now keep an LLM-based assistant open while they study. ChatGPT, Gemini, Copilot, and similar tools answer in plain language, remember the conversation, and rarely refuse a question. For a student trying to finish an assignment late at night, that combination is hard to compete with. Surveys across multiple campuses report that adoption is no longer a question of whether students use these tools, but of how often and for what [13].

This shift creates a problem the systems themselves were not designed to handle. Conversational LLMs were built to answer questions, not to teach. Their default behavior is to produce a complete, confident response in one turn, which removes the cognitive steps that learning research has shown are necessary for understanding to form [5, 22]. Students often report that they feel more confident after a study session with an LLM, yet their measured performance does not improve in the same way [19]. The convenience of the tool can make people feel they have learned something they have not.

In our earlier work we proposed and evaluated a different design point. *Study Assist* is an LLM-powered learning system that follows the three phases of self-regulated learning (SRL): forethought, performance, and self-reflection [25, 27]. Instead of generating a single long answer, the system breaks a topic into modules, each module into milestones, and walks the learner through them one at a time with a short teaching turn followed by a reflection question. A 55-participant evaluation on Prolific reported a System Usability Scale score of 77.99 (above the published average of 68), a significant pre-to-post gain in self-rated knowledge of Java programming, and 45 of 46 participants completing the assigned modules [1]. Participants also told us, repeatedly, that the structure mattered. Being able to see and modify the plan made the experience feel less like talking to a chatbot and more like working through a short course.

The same evaluation surfaced a structural gap. Study Assist had no place for the person who is normally responsible for what a student learns. A faculty member could not specify the scope of a topic, could not change how a quiz was graded, and could not see what their class was struggling with before lecture. The system was entirely student-facing in a setting that, in any normal classroom, is expert-mediated. Recent work makes the consequence explicit. Educators and developers see different harms from LLMs in education, and a tool that is comfortable for one is not automatically comfortable for the other [12]. Students, who are by definition novices in the subject they are studying, are also the least equipped to judge whether an AI response is appropriate for their course [9]. Simply telling them that the AI might be wrong does not solve the problem on its own [3, 15].

This report describes the second phase of the project, which addresses that gap. We re-frame the system around what we call the *Expert-in-the-Loop* (EitL) approach, a sociotechnical framing in which a designated domain expert defines, validates, and constrains the AI’s behavior on behalf of layman users. EitL is not unique to education. The same shape appears in medicine, where a physician validates AI recommendations before they reach a patient; in law, where an attorney reviews AI-generated analysis before it reaches a client; and in regulated finance, where an advisor checks AI portfolio suggestions before they reach a retail investor. In our case the expert is the course instructor and the layman user is the student, so we refer to the educational instantiation of EitL as the *Instructor-in-the-Loop* (IitL) configuration. The two terms are used together throughout the report: EitL names the broader framing, IitL names what we actually built and plan to study.

The independent study reported here covers four pieces of work that together set up the next phase of the research. First, we developed the EitL framing by reading across recent literature on appropriate reliance, the paradox of expertise, and educator-centered AI design, and by collecting open-text responses from five Kean University faculty about what they would need from a system like Study Assist. Second, we extended the system with a course-scoped data hierarchy (Course, CourseTopic, Module, Milestone), seven new agents that handle material ingestion, topic plan generation, two flavors of plan modification, instructor briefing, insights, and struggle summarization, and an analytics surface that exposes per-class progress to the instructor. Third, we built a synthetic-cohort simulator that drives twenty persona-tagged student accounts through a course over the public API, so that we can show an instructor a realistic mid-semester or end-of-semester dashboard without recruiting a live class. Fourth, we wrote the protocol for a planned think-aloud study with five to ten computer science faculty, which will be the empirical test of the design.

The rest of the report follows the same order. Section 2 reviews the literature and develops the EitL framing. Section 3 describes the redesigned system, including the new data model and agent architecture. Section 4 sets out the planned faculty study. Section 5 describes what we expect to find and how those findings will shape Phase 3. Section 6 closes with the independent-study scope and the immediate next steps.

2 Related Work and the Expert-in-the-Loop Framing

This section reviews the literature this project builds on and develops the framing that organizes the rest of the report. Four threads matter for our argument: LLM-based tutoring systems (Section 2.1), self-regulated learning with AI (Section 2.2), the question of whether students (and educators) can calibrate AI use on their own (Section 2.3), and the governance frameworks that increasingly require human oversight in high-stakes AI use (Section 2.4). Section 2.5 pulls these threads together into the Expert-in-the-Loop framing and explains how the educational case, Instructor-in-the-Loop, sits inside it.

2.1 LLM-Based Tutoring Systems and the Instructor Gap

A growing number of recent systems use LLMs to support learning rather than to answer one-off questions. LearnMate generates personalized learning plans along dimensions of goals, time, pace, and learning paths [24]; AutoPBL converts project tutorials into modular content with checkpoint questions and reported improved outcomes against self-learning [26]; SocraticAI uses guided questioning, retrieval grounding, and usage limits to push students toward problem decomposition, with 75 percent producing substantive reflections after two to three weeks [20]; CodeAid was deployed in a 700-student introductory programming course over twelve weeks and was designed to avoid direct code solutions and maintain transparency [14]. Lyu and colleagues ran a semester-long field study of LLM use in introductory CS [17], and Tzanoulinou and colleagues review the broader Socratic turn in LLM-based educators [23].

Two patterns stand out. First, all of these systems constrain the LLM in some pedagogical way: through structured plans, retrieval grounding, forced reflection, or restrictions on the kinds of answers it can produce. Second, the instructor is almost always treated as out-of-scope. LearnMate, AutoPBL, and SocraticAI provide no mechanism for course faculty to set the boundaries within which the AI operates. CodeAid recognizes the educator and shapes its design around their needs, but the day-to-day interaction is still student-to-AI. Our work fills exactly that role in a system that already has the SRL scaffolding from Phase 1 in place.

2.2 Self-Regulated Learning with AI

Self-regulated learning describes a cycle of forethought, performance, and self-reflection that learners use to manage their own studying [25, 27]. Several decades of work show that learners with stronger SRL skills do better academically and adapt better to new material. The challenge for educational technology has always been that the most effective SRL supports, such as modeling by a teacher or learning diaries and prompt cards, suffer from poor adherence and offer little real-time feedback when the learner is in the middle of a task.

Two recent papers connect SRL to generative AI directly. Prasad and Sane proposed a framework that maps the SRL phases onto specific roles for GenAI in CS instruction and used the framework to design interventions [18]. Li and colleagues extended this from design to deployment by showing that real-time analytics, when combined with GenAI, can be turned into adaptive scaffolds that target specific regulatory deficits such as poor planning or lack of reflection [16]. Phase 1 of our project sits in this space: Study Assist operationalizes forethought (mandatory onboarding), performance (milestone-based teaching with formative assessment at every milestone), and self-reflection (post-milestone reflection prompts and a longitudinal performance dashboard) inside a single system [1].

What is missing in this body of work is co-regulation. Research on SRL has long argued that novices often lack the metacognitive maturity to self-regulate effectively without an external regulator. In a classroom, that external regulator is the instructor. None of the SRL-with-GenAI systems above let the instructor play that role. This is a second reason to bring the instructor inside the system: not only a content-quality reason but also a learning-science reason.

2.3 Why Students Cannot Self-Calibrate Trust

A separate body of work asks whether the user of an LLM can decide on their own when to follow it and when to push back. The answer that comes out of this work is, mostly, no.

Bo and colleagues evaluated several interventions intended to help users calibrate their reliance on LLM outputs and found that the interventions help but are sensitive to wording and framing [4]. Kim and colleagues ran a 404-participant study with a fictional medical search tool and showed that

first-person uncertainty expressions such as “I’m not sure, but...” reduced over-reliance, while less specific expressions had weaker effects [15]. Bo and colleagues, in earlier work, looked at whether telling learners that the AI might be wrong changes how they interact with it; the effect is modest and uneven across users [3].

Chen and colleagues sharpen the picture in an educational tutoring decision task [9]. Novices followed AI suggestions more or less blindly, which raised accuracy when the AI was right and hurt accuracy when it was wrong. Experts overrode AI suggestions even when those suggestions were correct, which sometimes hurt their performance as well. The authors call this the paradox of expertise: the two groups that need help calibrating trust are exactly the groups least able to do that calibration on their own. Prather and colleagues report a related pattern from a different angle in their study of novice programmers using GenAI. Some students accelerated, others stalled, and the gap between the two groups widened over time [19].

The implication is that user-side trust calibration is necessary but not sufficient. A system that wants students to use it well cannot rely on warning labels alone. It needs structural intervention as well, and the obvious place to put that structure is the person who already has authority over the course content. A 2025 CHI Best Paper by Harvey and colleagues sharpens this point from the educator side: edtech developers tend to focus on technical harms such as hallucination, while educators see broader sociotechnical harms (inhibition of social development, increased workload, exacerbation of inequality), and call for educator-centered designs in which teachers can question, correct, and override AI outputs [12]. The instructor survey we describe in Section 3.1 aligns closely with this view.

2.4 Sociotechnical Governance and Cross-Domain Precedent

The case for keeping a human in the loop of an AI system is not only pedagogical. The CHI 2025 Sociotechnical AI Governance workshop argued that AI oversight has to extend beyond technical metrics into the institutional structures that surround the system, and that an individual human reviewer is rarely enough on their own [21]. The European Union’s AI Act now classifies AI systems used in education as high-risk and, in Article 14, requires that humans be able to understand the system’s behavior, monitor its operation, and override its outputs [10]. The instructor controls in our system are a direct match for this requirement: a published topic plan is editable, a quiz pattern is editable, and a published topic can be unpublished.

The same pattern shows up in other regulated domains. Studies of medical-information AI have shown that even fluent and confident outputs can be factually wrong, with inexperienced users particularly vulnerable to accepting them [2]. The structural similarity is the point. In medicine, in law, and in regulated finance, AI outputs that reach a layman user typically pass through a domain expert first. Education does not yet have a default version of this arrangement.

2.5 Synthesis: the Expert-in-the-Loop Framing

We collect the threads above into a single framing. Many AI systems operate inside sociotechnical environments where some stakeholder group is recognized as the domain expert and others are not. In those environments, the most effective deployments will be ones in which the expert constrains the AI’s output space, validates content before it reaches the layman user, and sets the standards by which the user’s interaction with that content is assessed. We call this the *Expert-in-the-Loop* (EitL) approach. Figure 1 shows the abstract pattern.

The same shape recurs across domains. Table 1 lists five instances. In each row, the layman user interacts with AI content that has already been shaped by an accountable expert. Without that shaping, the user has to do the calibration alone, and the work reviewed in Section 2.3 suggests they will mostly fail.

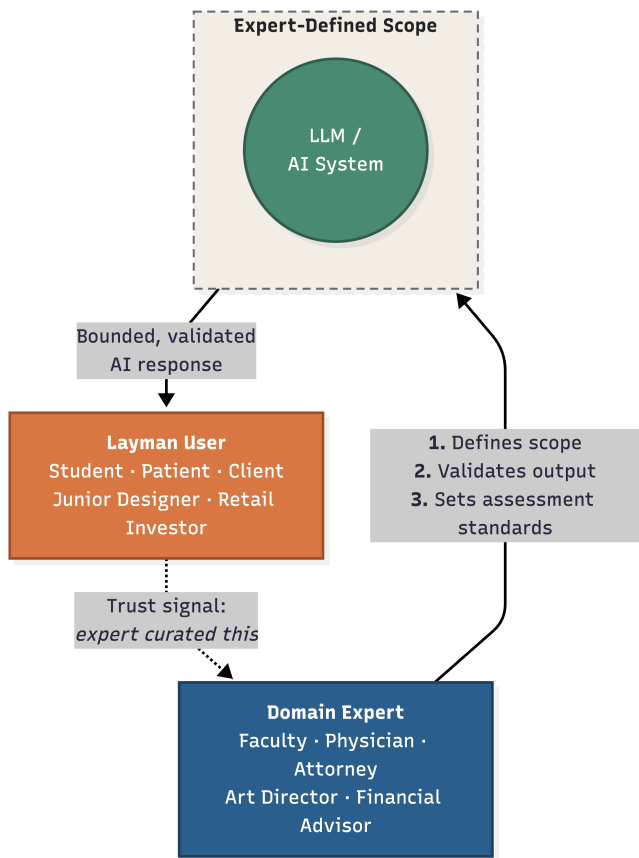


Fig. 1. The Expert-in-the-Loop (EitL) framing. In any sociotechnical domain in which a designated expert is held accountable for outcomes, the expert defines the AI’s scope, validates its outputs, and sets assessment standards on behalf of a layman user. Education is one instance of this broader pattern; the same shape recurs in medicine, law, creative work, and personal finance.

The educational case of EitL is the configuration we built and plan to study, and we refer to it as Instructor-in-the-Loop (IitL). In IitL the instructor uploads a syllabus and reference material, generates and edits topic plans drawn from that material, sets per-module quiz patterns that constrain the AI’s question generation, and monitors a class-level dashboard that surfaces aggregated struggle areas. Students enter through an enrollment code and only see published topics. Their experience inside a topic uses the same SRL flow validated in Phase 1, but the content scope and the assessment standards are now set by the instructor rather than inferred by the AI from a single topic name. The next section describes how we built that.

Table 1. The Expert-in-the-Loop pattern across five sociotechnical domains. The expert sets scope, validates output, and defines the standards for the layman user’s interaction with the AI.

Domain	Expert	Layman User	Expert’s role in the AI system
Education	Faculty / Instructor	Student	Defines curriculum scope, approves AI plans, sets quiz standards, monitors progress.
Medicine	Physician / Specialist	Patient	Validates AI diagnoses, sets treatment parameters, reviews patient-facing explanations.
Law	Attorney / Legal Scholar	Client / Citizen	Verifies AI legal analysis, constrains advice to jurisdiction, reviews compliance.
Creative	Art Director / Senior Designer	Junior Designer / Content Creator	Defines brand and style guidelines, approves AI-generated assets, shapes creative direction.
Finance	Financial Advisor (CFP)	Retail Investor	Validates AI portfolio suggestions, ensures regulatory compliance, personalizes risk tolerance.

3 System Design

This section describes the system we built in Phase 2. We summarize the four design themes that came out of an instructor survey at Kean (Section 3.1), describe the new course-scoped data hierarchy that organizes the work (Section 3.2), introduce the seven new agents that sit on top of the six existing pedagogical ones (Section 3.3), and explain the session-seeding and quiz-pattern mechanisms that connect the instructor side of the system back to the student-facing flow validated in Phase 1 (Section 3.4). Figure 2 shows the updated architecture.

3.1 From Instructor Survey to Design Requirements

Five computer science and technology faculty at Kean University answered an open-text survey in March 2026 about what they would need from a system like Study Assist. The survey was deliberately short (six questions, five to ten minutes per response) so we could collect targeted reactions without asking for a long commitment. We coded the responses by hand and four themes emerged.

The first theme is **pre-class preparation**. All five respondents identified some version of “students do not engage with the material before lecture” as a primary pain point. One Java instructor described teaching the basics of 2D arrays in one lecture and the advanced operations in the next, and noted that the second lecture only works if students practice the basics in between. Another respondent wanted a way to record whether students had actually done the assigned reading. The asked-for tool, in their language, was a short pre-class module with a few multiple-choice questions and a small amount of reading attached.

The second theme is **content alignment**. Respondents differed on how strict they wanted the system to be. Two preferred that the AI “strictly adhere to the terminology and scope provided in the syllabus,” one preferred that the AI “use AI capabilities to expand on topics and bring in outside examples within general parameters,” and two wanted both: a strict version for the assigned content and a more flexible version for optional enrichment. The takeaway is that scope control has to be a first-class instructor setting, not a hidden default.

The third theme is **grading metrics and incentives**. The respondents wanted the system’s data to be usable inside their existing course structures. The metrics most often mentioned were the number of attempts to reach 100 percent on a quiz, the time spent on each module, and

the timestamp of completion (in particular, whether the work was completed before class). One respondent wanted Canvas integration; another wanted the data tied to the participation grade.

The fourth theme is **dashboard requirements**. Every respondent wanted a class-level summary of which concepts students were struggling with before lecture, and most preferred narrative summaries over raw counts. Three of the five also wanted a way to see individual at-risk students and the most common wrong answers per question, not just which questions were missed most often.

These four themes drove the system design that follows. Pre-class preparation and content alignment together justify the new course-and-topic hierarchy; grading metrics and dashboard requirements together justify the new analytics service and the Insights and Struggle Summary agents.

3.2 New Hierarchy: Course, CourseTopic, Module, Milestone

The Phase 1 system stored each student’s session as a single Session document containing an inferred plan of two to five modules, each with three to six milestones. There was no concept of a course or a class. Phase 2 introduces three new collections that sit in front of the existing Session.

Course is an instructor-owned container. It has a unique six-character access code, an uploaded set of source files (syllabus and reference material), free-text global instructions, a plan strategy (week-based or module-based), and a persistent instructor chat used during topic generation.

CourseTopic is an instructor-defined topic that belongs to a course. Each topic has a status (draft, approved, published, or unpublished), an order index, and an array of modules. Each module carries its own quizPattern subdocument that constrains how the LLM generates the end-of-module quiz.

Enrollment links a student to a course, created when the student enters the access code. It also stores the student’s course-specific prior-knowledge responses for use in the teaching prompt.

A student’s individual learning thread on a topic is still a Session document. The Session now also carries optional courseId, courseTopicId, and enrollmentId references. When those references are present, the backend treats the session as instructor-mediated and overrides several defaults that otherwise apply (Section 3.4).

3.3 Seven New Agents on Top of the Existing Six

The new instructor-facing functionality is implemented as seven additional specialized prompt modules, all of which use the same baseAgent pattern as the Phase 1 modules. materialSummaryAgent takes the raw extracted text from the instructor’s uploaded files and produces a structured summary organized by chapter and concept. topicPlanGeneratorAgent takes that summary and the instructor’s plan strategy and proposes a set of CourseTopic drafts with full module and milestone trees; the output is checked by a topicPlanValidator that enforces the schema constraints (one to eight modules per topic, two to eight milestones per module, valid quiz-pattern structure). topicDraftModifyAgent rewrites unpublished drafts from a natural-language instruction such as “make module 3 more applied,” while courseTopicPlanModifyAgent handles edits to topics that have already been published, preserving identifiers so analytics continuity is not broken across edits. instructorBriefingAgent produces a short pre-lecture briefing in the instructor’s voice based on recent student activity in a chosen topic. instructorInsightsAgent and struggleSummaryAgent power the dashboard: the Insights agent produces longer narrative analyses across a topic or course, and the Struggle Summary agent produces short clusters of common misconceptions tied to specific milestones, intended for the instructor to skim before class.

These agents do not replace the Phase 1 modules. They run on the instructor side of the API and write to (or read from) the new collections. The student-facing flow inside a session continues to

use the same Phase 1 prompts, with the Teacher prompt enriched by the course’s global instructions and source-derived context.

3.4 Connecting the Instructor and Student Sides

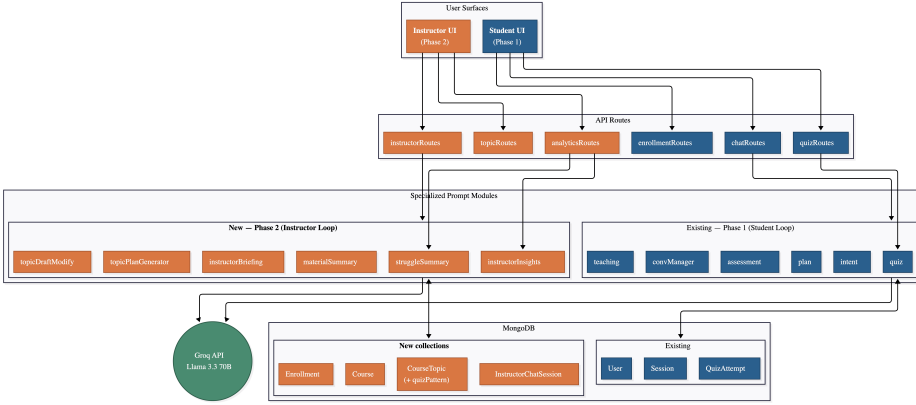


Fig. 2. Updated Study Assist architecture. Phase 1 (blue) contributed the student-facing surface and the six pedagogical prompt modules at the bottom of the agent layer. Phase 2 (orange) adds an instructor surface, four new API routes, seven new agents, and a course-scoped data hierarchy (Course → CourseTopic → Module → Milestone) that constrains every student-facing generation pass.

Two mechanisms connect the new instructor surface back to the student-facing flow. The first is *session seeding*. When a student opens a published topic for the first time, the backend creates a new Session and pre-populates it from the corresponding CourseTopic. This is implemented in `sessionSeedingService.js`. The seeded session enters directly into the learning phase, skipping the Intent Analyzer and Plan Generator steps that the Phase 1 student flow runs by default. The plan is copied from the topic’s modules and milestones, the active module is set to the first one, and the session is linked back to the course, topic, and enrollment.

This is where the instructor’s authority is enforced architecturally. The student does not see a free-form topic input on a course-linked session, because the topic is fixed by the blueprint. The Conversation Manager and Teacher prompts read the course’s global instructions and the student’s enrollment-specific prior-knowledge fields, so the same teaching prompt continues to produce appropriately personalized output without losing the instructor’s scope.

The second mechanism is *quiz-pattern propagation*. Each module on a CourseTopic carries a `quizPattern` with four fields: a question count, a weighted distribution over question types (conceptual, applied, recall, analytical), a difficulty mix (easy / medium / hard percentages), and a target Bloom’s cognitive level. There is also a free-text constraints field for instructor notes that do not fit into the structured fields. When a student finishes the milestones of a course-linked module and the system enters the quiz phase, the quiz route reads the pattern and passes it into the existing Quiz Generator prompt as explicit constraints (“produce 5 questions: 3 applied and 2 conceptual; difficulty mix is roughly 1 easy, 3 medium, 1 hard; target cognitive level is apply”). The generated quiz is therefore shaped by the instructor’s choices on a per-module basis, while the underlying agent code and the student-facing quiz-taking interface remain unchanged.

Figure 2 summarizes the result. The blue components are inherited from Phase 1; the orange components are new this semester. The data hierarchy on the right (Course, CourseTopic, Enrollment, InstructorChatSession) is what gives the instructor structural authority over what the student sees.

The agents in the upper part of the agent box are what give the instructor a usable interface for exercising that authority. The student-facing pieces in blue are unchanged in behavior; they now run inside boundaries that the instructor has set.

4 Method: Plan to Run the Study

This section describes the empirical study planned for the second half of the semester, which will be the first evaluation of the ItL configuration of Study Assist. The work this report covers is the protocol design and the supporting infrastructure that makes the study possible (in particular the synthetic-cohort simulator described in Section 4.4). Data collection itself is scheduled for the next phase. Figure 3 shows the planned workflow end to end.

4.1 Research Questions

The study is organized around three research questions, each tied to one of the contributions described in Section 3.

RQ1: How does an Expert-in-the-Loop architecture reshape the design requirements of an LLM-based educational system? Phase 1 produced design requirements that focused on the student’s interaction with the system (modify-the-plan controls, milestone pacing, reflection-question phrasing). We expect Phase 2 to surface a different set of requirements, centered on the boundaries the instructor sets around the AI rather than the AI’s interaction style with the student.

RQ2: What dimensions of expert oversight do instructors value when integrating AI tutoring into their courses? The survey responses described in Section 3.1 suggest that content alignment and assessment-rigor control will rank above raw analytics. RQ2 tests this in a setting where instructors are interacting with a working system rather than answering an open-text survey.

RQ3: How do instructors envision using AI-generated learning data to inform their teaching practice? The dashboard, the struggle summaries, and the briefing agent expose three different views of the same underlying data. RQ3 asks which of these views instructors actually use, and what they do with the information once they have it.

4.2 Participants

We will recruit five to ten faculty members from the Kean University Computer Science and Information Technology programs. The recruitment criterion is that the participant has taught an introductory programming course (CPS 1231 or an equivalent Java-based course) within the last three semesters. Compensation will follow the Phase 1 protocol structure: a one-hour session compensated at a professional rate.

4.3 Course Material and Scenarios

To keep results comparable across participants we fix the underlying course material rather than asking each participant to upload their own. The fixed material is a Java introductory syllabus structured into fifteen weekly units, which is also the syllabus that drives the synthetic-cohort recipes (Section 4.4). This mirrors the Phase 1 decision to assign “Java Programming” as the shared topic across participants, which made cross-participant comparison possible.

The study uses two within-subjects scenarios. The **mid-semester** scenario shows the system after the instructor has published seven of the fifteen weekly topics; the **full-semester** scenario shows it after all fifteen are published. The mid-semester scenario is the more diagnostically interesting one, because the synthetic cohort’s progress cap (Section 4.4) produces realistic stratification: strugglers stalled at topic 1, average students at topics 3–5, top students caught up at topic 7. The full-semester scenario gives the instructor a richer dashboard with more data to interpret. Scenario order will be counterbalanced across participants.

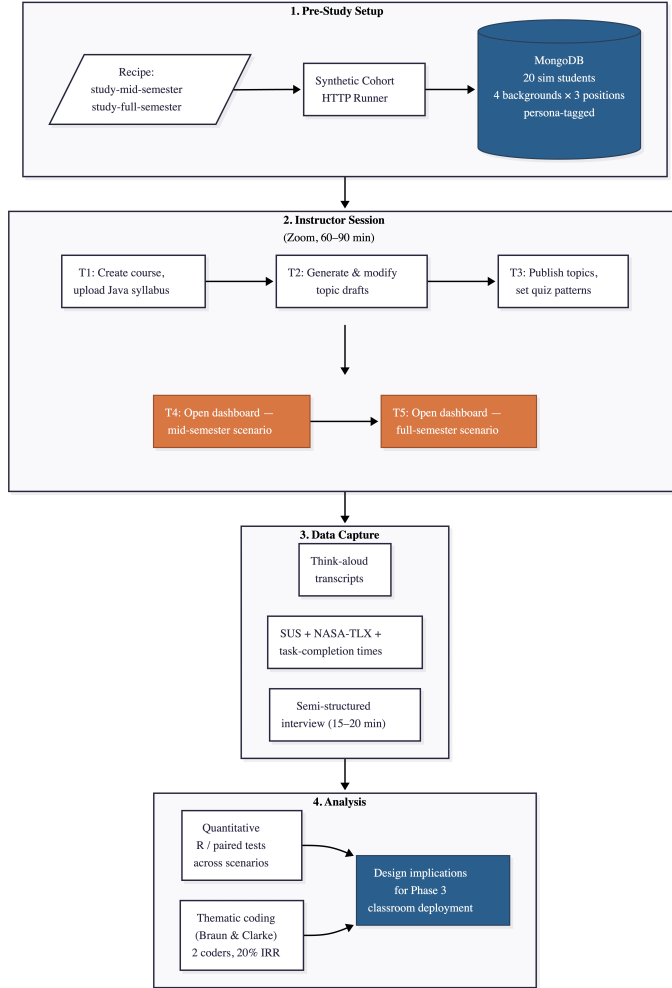


Fig. 3. Planned Phase 2 study workflow. A synthetic cohort of 20 simulated students is generated once and frozen, holding learner behavior constant across instructor sessions. Each instructor then completes five tasks across two within-subjects scenarios (mid-semester vs. full-semester), followed by a semi-structured interview. Quantitative measures (SUS, NASA-TLX, task times) and qualitative think-aloud and interview data converge on design implications for the Phase 3 classroom deployment.

4.4 Synthetic Cohort

Recruiting twenty real students per session would mix two kinds of variation we want to study separately: instructor sensemaking (which we are studying) and learner behavior (which we want fixed to a controlled distribution). We therefore drive each session against a synthetic cohort of twenty simulated students, generated through the public API by a runner that posts as if it were a browser; only the cohort generation and timestamp backdating talk to MongoDB directly.

Each synthetic student is the composition of one **background** (upper-year non-CS major, first-year with no programming exposure, first-year with substantial prior programming, or scripting-experienced) and one **position-in-class** (about-to-fail, average, or excellent). The background fixes

profile-page fields (major, programming exposure, motivation, self-confidence) and the student's preferred answer length; the position fixes a base probability of correct response, a retry policy, and a confusion rate. Chat replies are produced by a Groq-hosted instruct model with persona-specific prompts so the chat reads as plausibly student-written, while the quantitative signal that drives the dashboard (quiz scores, milestone outcomes, reflection counts) is driven by the gated probabilistic policy rather than the chat text. Each cohort is generated once with a fixed PRNG seed, snapshotted, and restored between sessions, so every participant sees the same dashboard data. Synthetic users carry a `profile.isSynthetic` flag and an analytics filter excludes them from production metrics by default.

4.5 Procedure and Tasks

Each session is run on Zoom with screen share enabled and lasts sixty to ninety minutes. The session has three parts.

Pre-task survey (5 minutes). Demographics, current pre-class preparation practice, prior experience with LLM-based tools in teaching, and the GenderMag-style cognitive-style fields also used in Phase 1 [8].

Think-aloud session (45–60 minutes). Participants complete five tasks. T1, T2, and T3 happen once at the start of the session; T4 and T5 happen once per scenario.

- **T1.** Create a new course, upload the provided Java syllabus, and set the global instructions and plan strategy.
- **T2.** Generate topic drafts and modify two of them through the natural-language modify interface. Set per-module quiz patterns for one of the modified topics.
- **T3.** Approve and publish the topics; locate the access code.
- **T4.** (Mid-semester scenario.) Open the dashboard. Identify which students are currently at risk, identify the concept the class is most often struggling with, and describe in their own words what they would change in the next lecture.
- **T5.** (Full-semester scenario.) Repeat T4 on the end-of-semester dashboard.

Participants are asked to think aloud throughout the tasks. The facilitator follows a fixed prompt-script for clarification and only intervenes when a participant has been silent for more than thirty seconds.

Semi-structured interview (15–20 minutes). Eight questions covering: how the participant would introduce the system to their own students; what conditions would have to be in place before they would use it in a real course; what additional control mechanisms they would want; how often they noticed AI errors or hallucinations during the session; what they would change about the dashboard; and how they would tie the data into their existing grading structure.

4.6 Measures

Quantitative measures are the System Usability Scale [7], the NASA-TLX workload questionnaire [11], per-task completion time from the screen capture, and the count and type of plan modifications the participant made during T2. Dashboard interpretation accuracy will be coded after the session from the participant's free-text answers in T4 and T5, compared against the ground-truth manifest produced when the synthetic cohort was generated. Qualitative data consists of the think-aloud and interview transcripts. The full analysis plan, including the procedure for each step, is described in Section 5. The two scenarios (mid-semester and full-semester) are within-subjects with counterbalanced order.

4.7 Ethics and Data Handling

The Phase 2 protocol is an amendment to the Kean University IRB protocol used for Phase 1. Participation is voluntary; the participant may end the session at any time. Recordings are stored on an encrypted Kean-managed drive, transcripts are de-identified before coding, and any direct quote used in publication will be reviewed with the participant beforehand. Synthetic-cohort data carries no personally identifying information; the names sampled by the simulator are drawn from a curated diversity-balanced pool and are never assigned to real users.

Read end to end, Figure 3 captures the full pipeline. Cohort generation runs once before the study window opens and produces a snapshot that every participant session restores from, so each instructor sees the same simulated dashboard. Each session then feeds the analysis pipeline with three streams of data (think-aloud audio, screen capture, and questionnaire scores), and the interview adds a fourth. The output of the analysis is a set of design implications that lock in the configuration to be tested in the Phase 3 classroom deployment with real students.

5 Analysis Plan

The empirical work described in Section 4 will produce data that has not yet been collected. This section describes how that data will be analyzed once it is in hand. The section is forward-looking by design and does not contain results.

5.1 Quantitative Analysis

The quantitative measures listed in Section 4.6 will be analyzed in two passes inside R using RStudio, mirroring the Phase 1 pipeline.

The first pass is descriptive. We compute means, standard deviations, and per-participant distributions for SUS, the six NASA-TLX subscales, per-task completion time, plan-modification counts, and dashboard interpretation accuracy across the full sample. The output is the summary table that anchors the participants section of the eventual paper.

The second pass is the within-subjects scenario contrast (mid-semester vs full-semester). For each measure, we run a paired Wilcoxon signed-rank test, which is appropriate for the small sample size ($n = 5$ to 10) where the normality assumption of the paired t -test cannot be relied on. Three contrasts are pre-specified: (a) SUS for the instructor surface against the Phase 1 student-facing baseline of 77.99; (b) NASA-TLX mental demand on the first-time setup tasks (T1 and T2) against the dashboard interpretation tasks (T4 and T5); and (c) per-task completion time on the full-semester scenario against the mid-semester scenario. Effect sizes (matched-pairs rank biserial correlation) are reported alongside p -values. Spearman's ρ is computed between the participant's prior AI experience and their willingness to delegate scope decisions to the system.

5.2 Qualitative Analysis

Think-aloud audio and interview audio are transcribed verbatim and de-identified before any coding begins. Coding is performed in NVivo following the Braun and Clarke six-phase reflexive thematic analysis procedure: familiarization, initial coding, theme generation, theme review, theme definition, and write-up [6].

Two coders work independently on a 20 percent subsample of the transcripts to produce an initial codebook. Inter-rater reliability on that subsample is computed using Cohen's κ , with a target of $\kappa \geq 0.7$ before the remaining transcripts are divided between the two coders for single-coding. Disagreements that surface during single-coding are flagged and adjudicated against the codebook before being included in the final theme set.

Three thematic lenses, drawn from the literature in Section 2 and the instructor survey in Section 3.1, are pre-specified for the theme review phase: *authority as legitimacy* (whether instructors frame their oversight as professional standing rather than only as quality control), *trust transfer* (whether instructors hypothesize that students will trust the AI more when they know an instructor curated it), and *the workload trade-off* (whether instructors value the AI-generated topic drafts but worry about the time cost of reviewing them). Themes that emerge inductively from the data are still admitted; the lenses are a prior, not a constraint.

5.3 Triangulation and Reporting

The two analysis streams are triangulated next. Each qualitative theme is checked for consistency against the quantitative pattern in the same area, and quantitative findings without a qualitative interpretation (and vice versa) are flagged for further investigation.

The synthesized output is a single deliverable: a list of design implications for the Phase 3 classroom deployment. Each implication is tagged with the research question it addresses (RQ1, RQ2, or RQ3 from Section 4.1), the dimension of evidence (quantitative, qualitative, or both) that supports it, and the strength of that support. Features that score well on both streams are locked in for Phase 3; features that score poorly on either stream are revisited or removed before any real students see them. The list is the artifact that hands off from this independent study to the Phase 3 work.

5.4 Threats to Validity

Three threats are worth naming. The first is the realism of the synthetic cohort. Replies are produced by an LLM with persona-specific prompts and are fluent and in character, but the simulated students do not internalize content across turns the way a real student would. We mitigate this by driving the dashboard signal from the gated probabilistic policy rather than the chat text, but the chat snippets the instructor reads during a session are still the weakest link in the realism chain.

The second threat is single-institution sampling. All participants will be Kean faculty teaching CS, and the design implications we extract may not transfer to other institutions or other disciplines. We chose this trade-off deliberately for a first instructor evaluation, because it lets us ground the recruitment in an existing relationship and run sessions in person where useful. Phase 3 will broaden this.

The third threat is self-report on hypothetical adoption. Asking instructors what they would do with a system in their own course is not the same as observing what they actually do. The think-aloud protocol gives us behavior on the study tasks, but the questions about real-classroom adoption are predictive at best. The Phase 3 deployment is the answer to that limitation.

6 Conclusion and Next Steps

This report covered the second phase of a research program that began with Study Assist, an LLM-powered learning system grounded in self-regulated learning and validated in a 55-participant Phase 1 evaluation. The Phase 1 results were encouraging on both usability and learning outcomes, but the same evaluation surfaced a structural gap: the system had no role for the instructor, who is normally the person responsible for what a student learns.

Phase 2 reframes the work around the Expert-in-the-Loop approach, in which a designated domain expert defines, validates, and constrains AI behavior on behalf of layman users. The educational instantiation, which we call Instructor-in-the-Loop, is what we built and plan to study. The independent study reported here covers four pieces of work that together set up that study. The first is the EitL framing itself, drawn from a literature review and an open-text survey of five Kean University faculty. The second is an extended system architecture with a course-scoped

data hierarchy and seven new agents that give the instructor structural authority over what the AI generates. The third is a synthetic-cohort simulator that produces a realistic mid-semester or end-of-semester dashboard for a fixed twenty-student cohort. The fourth is the protocol for a planned think-aloud study with five to ten computer science faculty.

The empirical work itself is scheduled for the months following this submission. The output of that study will be a locked configuration that the Phase 3 classroom deployment evaluates against real students. The independent-study scope ends here: design and protocol now, execution next.

References

- [1] Iyadunni J. Adenuga and Md Jonayed Hossain Chowdhury. 2026. Guided Self-Regulated Learning via LLMs. In *Proceedings of the 28th International Conference on Human-Computer Interaction (HCII 2026)*. Springer. Phase 1 of this research program..
- [2] I. Amaro, A. Della Greca, R. Francese, G. Tortora, and C. Tucci. 2023. AI Unreliable Answers: A Case Study on ChatGPT. In *International Conference on Human-Computer Interaction (HCII 2023)*. Springer, 23–40.
- [3] J. Y. Bo, H. Kumar, M. Liut, and A. Anderson. 2024. Disclosures & Disclaimers: Investigating the Impact of Transparency Disclosures and Reliability Disclaimers on Learner-LLM Interactions. In *Proceedings of the AAAI Conference on Human Computation and Crowdsourcing (HCOMP)*, Vol. 12. 23–32.
- [4] J. Y. Bo, S. Wan, and A. Anderson. 2025. To Rely or Not to Rely? Evaluating Interventions for Appropriate Reliance on Large Language Models. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. Honourable Mention..
- [5] C. Brame. 2016. Active Learning. Vanderbilt University Center for Teaching.
- [6] V. Braun and V. Clarke. 2006. Using Thematic Analysis in Psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101.
- [7] J. Brooke. 1996. SUS: A Quick and Dirty Usability Scale. *Usability Evaluation in Industry* 189, 194 (1996), 4–7.
- [8] M. Burnett, S. Stumpf, J. Macbeth, S. Makri, L. Beckwith, I. Kwan, A. Peters, and W. Jernigan. 2016. GenderMag: A Method for Evaluating Software’s Gender Inclusiveness. *Interacting with Computers* 28, 6 (2016), 760–787.
- [9] E. Chen, J. Li, S. Huang, X. Tang, J. Lin, P. Carvalho, and K. Koedinger. 2026. AI Knows Best? The Paradox of Expertise, AI-Reliance, and Performance in Educational Tutoring Decision-Making Tasks. In *Proceedings of the 16th International Learning Analytics and Knowledge Conference (LAK 2026)*. doi:10.1145/3785022.3785035
- [10] European Union. 2024. Regulation (EU) 2024/1689 (EU AI Act), Article 14: Human Oversight Requirements for High-Risk AI Systems. Official Journal of the European Union.
- [11] S. G. Hart and L. E. Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research. In *Advances in Psychology*. Vol. 52. 139–183.
- [12] B. Harvey, D. Murthy, S. Mahmood, K. Holstein, and D. W. Shaffer. 2025. “Don’t Forget the Teachers”: Towards an Educator-Centered Understanding of Harms from Large Language Models in Education. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. doi:10.1145/3706598.3713210 Best Paper Award..
- [13] H. Jo. 2024. From Concerns to Benefits: A Comprehensive Study of ChatGPT Usage in Education. *International Journal of Educational Technology in Higher Education* 21, 1 (2024), 35.
- [14] Majeed Kazemitabaar, Runlong Ye, Xiaoning Wang, Austin Z. Henley, Paul Denny, Michelle Craig, and Tovi Grossman. 2024. CodeAid: Evaluating a Classroom Deployment of an LLM-based Programming Assistant that Balances Student and Educator Needs. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. doi:10.1145/3613904.3642398
- [15] S. S. Kim, Q. V. Liao, M. Vorvoreanu, S. Ballard, and J. W. Vaughan. 2024. “I’m Not Sure, But...”: Examining the Impact of Large Language Models’ Uncertainty Expression on User Reliance and Trust. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*. 1809–1823.
- [16] T. Li, D. Nath, Y. Cheng, Y. Fan, X. Li, M. Raković, H. Khosravi, Z. Swiecki, Y. S. Tsai, and D. Gašević. 2025. Turning Real-Time Analytics into Adaptive Scaffolds for Self-Regulated Learning Using Generative Artificial Intelligence. In *Proceedings of the 15th International Learning Analytics and Knowledge Conference (LAK 2025)*. 667–679.
- [17] W. Lyu, Y. Wang, T. Chung, Y. Sun, and Y. Zhang. 2024. Evaluating the Effectiveness of LLMs in Introductory Computer Science Education: A Semester-Long Field Study. In *Proceedings of the Eleventh ACM Conference on Learning at Scale (L@S ’24)*. doi:10.1145/3657604.3662036
- [18] P. Prasad and A. Sane. 2024. A Self-Regulated Learning Framework Using Generative AI and Its Application in CS Educational Intervention Design. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*. 1070–1076.

- [19] J. Prather, B. N. Reeves, J. Leinonen, S. MacNeil, A. S. Randrianasolo, B. A. Becker, B. Kimmel, J. Wright, and B. Briggs. 2024. The Widening Gap: The Benefits and Harms of Generative AI for Novice Programmers. In *Proceedings of the 2024 ACM Conference on International Computing Education Research (ICER)*. 469–486.
- [20] K. Sunil and A. Thakkar. 2025. SocraticAI: Transforming LLMs into Guided CS Tutors Through Scaffolded Interaction. [arXiv:2512.03501](https://arxiv.org/abs/2512.03501)
- [21] H. Suresh, K. Holstein, J. W. Vaughan, et al. 2025. Sociotechnical AI Governance (STAIG) Workshop. CHI 2025 Workshop.
- [22] J. Sweller. 1988. Cognitive Load During Problem Solving: Effects on Learning. *Cognitive Science* 12, 2 (1988), 257–285.
- [23] D. Tzanoulinou, L. Triantafyllopoulos, E. Paxinou, G. Feretzakis, D. Kalles, and V. Verykios. 2025. The Socratic Turn in AI: Reviewing the Transformation of LLMs into Dialogical Educators. In *Proceedings of the 16th International Conference on Information, Intelligence, Systems and Applications (IISA)*. IEEE.
- [24] X. J. Wang, C. P. Lee, and B. Mutlu. 2025. LearnMate: Enhancing Online Education with LLM-Powered Personalized Learning Plans and Support. In *Extended Abstracts of the 2025 CHI Conference on Human Factors in Computing Systems*. [doi:10.1145/3706599.3719857](https://doi.org/10.1145/3706599.3719857)
- [25] P. H. Winne and A. F. Hadwin. 1998. Studying as Self-Regulated Learning. In *Metacognition in Educational Theory and Practice*. 277–304.
- [26] Y. Zhu, Z. Ye, Y. Yuan, W. Tang, C. Yu, and Y. Shi. 2025. AutoPBL: An LLM-Powered Platform to Guide and Support Individual Learners Through Self Project-Based Learning. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. [doi:10.1145/3706598.3714261](https://doi.org/10.1145/3706598.3714261)
- [27] B. J. Zimmerman. 2002. Becoming a Self-Regulated Learner: An Overview. *Theory Into Practice* 41, 2 (2002), 64–70.